

Disadvantages of CQRS

For systems that simply need basic CRUD operations, implementing a CQRS system may introduce unnecessary complexity. CQRS systems have a higher degree of complexity, especially when combined with event-sourcing. For this reason, it is important to understand that CQRS is not applicable to all situations. Software architects should be aware that CQRS does not have to be applied to the entirety of a software system. It can be applied to just some subsystems of a larger architecture where it will be of the greatest benefit.

While using different data stores for query and command models can improve performance and increase security, you do have to consider that when you perform read operations, you may be reading data that is stale.

If separate databases are used in a CQRS system, the database used for reads must be kept up to date with the database used for writes. The software system will follow an *eventual consistency* model where if no new updates are made to a given item, eventually all access to that item will acquire the latest data.

Whether the system is using event-sourcing or some other mechanism to keep the two in sync, there will be some time delay, even if it is a small one, before they are made consistent. This means that any data that is read could potentially be stale if the latest updates to it have not been applied yet. This is in contrast with a *strong consistency* model, in which all data changes are atomic and a transaction is not allowed to complete until all of the changes have been completed successfully or, in the case of a failure, everything has been undone.